



Comparativa: Exploración de modelos de inteligencia artificial en procesos de diseño de materiales.

Proyecto Integrador

Patrocinadores:

Dr. Juan Arturo Nolazco Flores

Dr. Samuel Lara Ávila

Integrantes:

Javier González Aces - A01274650

José Alejandro Cortés Pérez - A01795033

Osiris Xcaret Saavedra Solís - A01795992

Fecha de entrega: 08 de febrero del 2026

Introducción.....	3
NVIDIA ALCHEMI Toolkit-Ops	4
DifCSP	6
Experimentación con MatterGen y MatterSim	9
Generación de estructuras cristalinas.....	9
Evaluación energética y modelo de fuerzas	9
Relajación estructural.....	9
Relajación estática ($T = 0\text{ K}$)	9
Dinámica molecular a temperatura finita	10
Annealing térmico	10
Filtrado y selección de candidatos	10
Análisis y visualización.....	10
Exportación para cálculos ab initio	10

Introducción

En este trabajo se experimentó con distintos modelos para las etapas de generación y simulación. Se realizó un trabajo de exploración de capacidades de los siguientes:

Modelo	Capacidad
MatterGen	Generación
MatterSim	Simulación
DiffCSP	Generación
NVIDIA Alchemi	Simulación

En el caso de MatterGen y MatterSim, se estudió el pipeline desarrollado por un equipo de trabajo de un trimestre anterior, estudiando así aspectos de implementación al igual que las capacidades de los modelos en sus versiones base.

NVIDIA ALCHEMI Toolkit-Ops

Es un modelo de simulación para computación dinámica acelerada con GPU que permite la reconstrucción de los vecindarios de células así como en contra la dispersión de las correlaciones y otras características propias de los átomos con la finalidad de estudiar cómo cambian dichas propiedades al cambiar parámetros o hacer cambios a nivel químico por otra parte reconstruir algoritmos por medio de este modelo permite un estudio más detallado sobre las propiedades de distintos materiales así como las propiedades que podrían ser entendidas y estudiadas ya sean en conjunto o por separado.

Por otra parte, dentro de las capacidades principales del modelo se encuentra el estudio de los vecindarios celulares, así como calcular la electrostática y el procesamiento por batch, dentro de estos componentes incluye el procesamiento nativo de tensores por medio de pytorch es su respectiva compilación todo esto por medio de la adaptación con el sistema de CUDA e integración con Python.

A continuación, se integra una serie de los ejemplos infracciones descritas por la librería con la finalidad de estudiarlo por separado y después integrar los procesos más complejos.

Se observa en este ejemplo el cálculo de DFT-D3 que es una variante de la dispersión de London, pero con corrección.

Con la siguiente línea importamos el módulo que permite estudiar los átomos que se encuentran cerca:

```
from nvalchemiops.neighborlist.neighborlist import neighbor_list
```

Para cargar los archivos que son en formato XYZ y que permitirán el puente entre distintos programas es posible hacerlo por medio de una lectura de líneas como lo muestra la librería a continuación:

```
with open("./dimer.xyz") as f:
```

```
    lines = f.readlines()
```

```
    num_atoms = int(lines[0])
```

```
    coords = torch.zeros(num_atoms, 3, device=device, dtype=dtype)
```

```
    atomic_numbers = torch.zeros(num_atoms, device=device, dtype=torch.int32)
```

La computación de la lista de los vecindarios permite que se puedan analizar cómo interactúan las fuerzas entre sí, las tensiones y las energías dentro de los átomos cercanos, para ello se ingresan varios parámetros al modelo los cuales son demostrados por medio del siguiente código incluido en el toolkit.

```
neighbor_matrix, num_neighbors_per_atom, neighbor_matrix_shifts = neighbor_list(  
    coords,  
    cutoff=20.0, # Interaction cutoff in Angstrom
```

```

    cell=cell,

    pbc=pbc,

    method="cell_list", # O(N) cell list algorithm

    max_neighbors=64, # Maximum neighbors per atom

)

```

Una vez que has sido calculada la matrix de los vecindarios se pueden establecer las variables que resultan en la energía y la fuerza Todo esto por medio de la inyección de 3 parámetros previos como se marca a continuación han dado documentación:

```

energies, forces, coord_num = d3_module(

    positions=coords,

    numbers=atomic_numbers,

    neighbor_matrix=neighbor_matrix,

)

```

Para el procesamiento de cristales los pasos son similares en los cuales se incluyen parámetros como el numero de estructuras a procesar el cutoff en Angstrom:

```

BATCH_SIZE = 16 # Number of structures to process simultaneously

CUTOFF = 25.0 # Interaction cutoff in Angstrom

```

DifCSP

DifCSP (Diffusion-based Crystal Structure) es un modelo generativo por difusión orientado a la predicción y a la síntesis de estructuras cristalinas. Los modelos generativos por diffusion aprenden a modelar una distribución de datos complejos mediante un proceso de ruido progresivo y su posterior reconstrucción. Para DifCSP, el modelo aprende la distribución conjunta de posiciones atómicas, tipos de átomos (números atómicos), parámetros de la celda cristalina y simetrías y periodicidad del cristal. El objetivo principal del modelo es generar estructuras cristalinas físicamente plausibles que posteriormente puedan ser evaluadas mediante una simulación física.

El proceso de DifCSP puede dividirse en dos fases principales:

Difusión directa: Se añade ruido gaussiano de manera progresiva a las posiciones atómicas y parámetros cristalinos, transformando una estructura ordenada en una distribución cercana al ruido puro, este proceso se controla mediante un conjunto de pasos temporales discretos.

Difusión inversa: Se remueve gradualmente el ruido para reconstruir una estructura cristalina coherente. Es durante esta fase que el modelo captura regularidades físicas, tales como distancias interatómicas factibles, coordinación local y restricciones geométricas del cristal.

El modelo DifCSP se apoya en redes neuronales profundas, procesamiento masivo con GPU y el uso de tensores y frameworks de PyTorch.

Para la importación de módulos y la carga de estructuras cristalinas (CIF) se utiliza el código en python:

```
from pathlib import Path
import numpy as np
import torch
from pymatgen.core import Structure

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
dtype = torch.float32

def load_cif_as_tensors(cif_path: str):
    s = Structure.from_file(cif_path)

    frac_coords = torch.tensor(s.frac_coords, dtype=dtype)
    atomic_numbers = torch.tensor([site.specie.Z for site in s.sites],
                                   dtype=torch.long)
    lattice = torch.tensor(s.lattice.matrix, dtype=dtype) # 3x3

    return frac_coords, atomic_numbers, lattice

# Ejemplo: cargar un folder con CIFs
cif_dir = Path("./cifs")
cif_files = sorted(list(cif_dir.glob("*.cif")))
```

```

frac_list, z_list, lat_list = [], [], []
for fp in cif_files[:10]:
    fc, z, lat = load_cif_as_tensors(str(fp))
    frac_list.append(fc)
    z_list.append(z)
    lat_list.append(lat)

print("Ejemplo #estructuras:", len(frac_list))
print("N átomos estructura 0:", frac_list[0].shape[0])

```

DifCSP (y en general las GNNs) necesitan vecindarios, que se pueden calcular mediante un PBC práctico con ASE:

```

from ase import Atoms
from ase.neighborlist import neighbor_list as ase_neighbor_list

def build_neighbor_list(frac_coords, atomic_numbers, lattice, cutoff=6.0):
    # Convert frac -> cart
    cart = frac_coords @ lattice # (N,3)

    atoms = Atoms(
        numbers=atomic_numbers.cpu().numpy(),
        positions=cart.cpu().numpy(),
        cell=lattice.cpu().numpy(),
        pbc=True
    )

    i, j, S = ase_neighbor_list("ijS", atoms, cutoff)
    # S es el shift de celda en enteros (n1,n2,n3) para la imagen periódica

    # Distancias (reconstruyendo imagen periódica)
    # rj_image = rj + S @ cell
    cell = lattice.cpu().numpy()
    ri = atoms.positions[i]
    rj = atoms.positions[j] + (S @ cell)
    d = np.linalg.norm(rj - ri, axis=1)

    edge_index = torch.tensor(np.stack([i, j], axis=0), dtype=torch.long)
    shifts = torch.tensor(S, dtype=torch.long)
    distances = torch.tensor(d, dtype=dtype)

    return edge_index.to(device), shifts.to(device), distances.to(device)

# Demo
fc0, z0, lat0 = frac_list[0].to(device), z_list[0].to(device),
lat_list[0].to(device)
edge_index, shifts, distances = build_neighbor_list(fc0, z0, lat0, cutoff=6.0)
print(edge_index.shape, shifts.shape, distances.shape)

```

Los siguientes pasos, abarcan la difusión directa (forward process) sobre las coordenadas fraccionarias y la diffusion inversa (reverse diffusion). Posteriormente, se realiza el entrenamiento del modelo y se genera o reconstruye una estructura cristalina desde el ruido generado, para finalmente realizar una validación que incluye:

- Distancias interatómicas viables.
- Valencias factibles.
- Estimación de energía.

Experimentación con MatterGen y MatterSim

Generación de estructuras cristalinas

La generación de estructuras cristalinas se realizó utilizando MatterGen, un modelo generativo basado en difusión entrenado para la síntesis ab initio de cristales periódicos. Las estructuras se generaron condicionando explícitamente el sistema químico, definido como el conjunto de elementos presentes (por ejemplo, Si-C), lo que garantiza que todas las configuraciones propuestas respeten la composición elemental deseada.

Para cada sistema químico se generaron múltiples candidatos de manera simultánea mediante muestreo por lotes. Se empleó guidance por difusión (classifier-free guidance) con un factor de guía ajustable, lo que permite sesgar el proceso generativo hacia regiones del espacio estructural más físicamente plausibles sin imponer restricciones geométricas explícitas. Las estructuras generadas se exportaron en formato .xyz para su posterior procesamiento.

Tras la generación, se verificó automáticamente la composición química de cada estructura, descartando aquellas que no coincidieran exactamente con el conjunto de elementos especificado.

Evaluación energética y modelo de fuerzas

La evaluación energética y el cálculo de fuerzas atómicas se llevaron a cabo mediante MatterSim, un potencial interatómico basado en aprendizaje automático. Este modelo actúa como sustituto eficiente de métodos ab initio, permitiendo explorar el paisaje energético de manera computacionalmente eficiente. MatterSim se utilizó tanto para optimización geométrica como para simulaciones de dinámica molecular, y fue ejecutado indistintamente en CPU o GPU según la disponibilidad del hardware.

Relajación estructural

Las estructuras generadas se sometieron a distintos esquemas de relajación, seleccionables según el objetivo del estudio:

Relajación estática ($T = 0$ K)

En el caso de relajación a temperatura cero, se realizó una optimización geométrica estándar utilizando el algoritmo FIRE, hasta que la fuerza máxima sobre cualquier átomo descendió por debajo de un umbral predefinido. Este procedimiento permite localizar mínimos locales del paisaje energético a 0 K.

Dinámica molecular a temperatura finita

Para estudiar el comportamiento a temperatura finita, se llevaron a cabo simulaciones de dinámica molecular (MD) con un termostato de Langevin. Las velocidades iniciales se asignaron según una distribución de Maxwell–Boltzmann correspondiente a la temperatura objetivo. Durante la simulación se monitorizaron la energía total y la temperatura instantánea, y se almacenaron trayectorias completas para análisis posterior.

Annealing térmico

Adicionalmente, se implementó un protocolo de annealing, en el cual las estructuras se sometieron a un enfriamiento gradual desde una temperatura inicial elevada hasta 0 K. Este proceso se realizó mediante ciclos cortos de dinámica molecular a temperaturas decrecientes, seguidos de una relajación final estática. Este enfoque facilita la superación de barreras energéticas y reduce la dependencia de las configuraciones finales respecto a condiciones iniciales desfavorables.

Filtrado y selección de candidatos

Una vez relajadas, las estructuras se evaluaron según criterios físicos mínimos. En particular, se exigió que la fuerza máxima residual y la distancia interatómica mínima cumplieran umbrales predefinidos, descartando configuraciones no físicas o insuficientemente convergidas. Las estructuras aceptadas se ordenaron según su energía por átomo, y se seleccionaron los candidatos de menor energía para análisis posterior.

Análisis y visualización

El pipeline genera automáticamente un conjunto de gráficos de resumen que incluyen: energía por átomo, distribución de energías, relación entre energía y fuerza residual, y distancias interatómicas mínimas. Estos gráficos permiten una inspección rápida de la calidad y diversidad de los candidatos generados.

Asimismo, se almacena un archivo resumen en formato tabular que contiene las propiedades estructurales y energéticas de todos los candidatos considerados.

Exportación para cálculos ab initio

Como paso final, las estructuras de menor energía se exportaron a archivos de entrada para Quantum ESPRESSO, permitiendo su validación mediante cálculos de teoría del funcional de la densidad (DFT). Los puntos k se seleccionaron automáticamente en función de la periodicidad del sistema (0D, 2D o 3D), y se asignaron pseudopotenciales apropiados para cada elemento.